

Atividade de AEDS III - Ordenação Ineficiente

Objetivo

Implemente um programa em C ou C++ que faça a ordenação de um vetor de qualquer tamanho. O algoritmo deverá aprimorar a técnica do **Insertion sort** para ordenar dois elementos por vez.

Instruções

1. **Implemente a função void insertionSortDuplo(int vetor[], int n)**
 - a. Essa função recebe como entrada uma lista de inteiros em qualquer ordem e altera a posição dos elementos de forma que eles fiquem ordenados. Essa implementação deve conseguir resolver o problema usando a técnica do Insertion sort para ordenar dois elementos por vez.
 - b. Ao fim de cada passo do laço externo, dois elementos deverão ter sido ordenados, ou seja, a cada iteração do laço externo você deverá avançar duas posições e colocar os dois elementos de trabalho na sublista ordenada.
2. **Descrição do algoritmo - a cada iteração, o algoritmo deve:**
 - a. Retirar os dois próximos elementos do vetor não ordenado (i e $i+1$)
 - b. Compará-los entre si e identificar qual é o menor e qual é o maior
 - c. Inserir primeiro o menor na posição correta do subvetor já ordenado (o subvetor ordenado cresceu em um elemento)
 - d. Inserir depois o maior na posição correta do subvetor já ordenado
 - e. Avançar i em 2 posições
3. **Análise a complexidade das funções:**
 - a. **Análise a notação Big Oh para esta implementação.** Explique como o tamanho da lista de entrada afeta o número de operações e se e como o uso desta técnica aprimorada pode ajudar a reduzir a complexidade do algoritmo de ordenação.
4. **Teste seu código:**
 - a. Crie vetores com diferentes tamanhos e organizações.
 - b. Discuta quanto essa otimização pode acelerar o processo de ordenação do vetor.
5. **Exemplo:**

Vetor inicial: [7, 3, 5, 1, 4, 2]

Iteração	Par retirado	Menor / Maior	Subvetor ordenado após inserção
1	(7, 3)	3 / 7	[3, 7]

2	(5, 1)	1 / 5	[1, 3, 5, 7]
3	(4, 2)	2 / 4	[1, 2, 3, 4, 5, 7]